

IF330 – Pemrograman Web

10 - Migrations & Models

Fenina Adline Twince Tobing

I Putu Ananta Yogiswara

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA
SEMESTER GANJIL TAHUN AJARAN 2022/2023

REVIEW

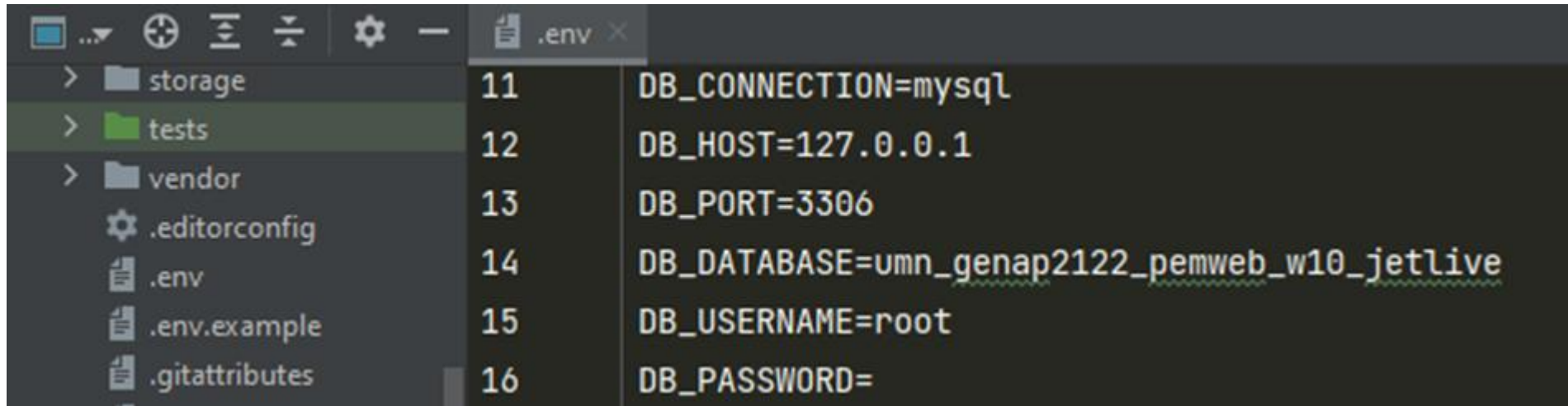


- Controller
- Routing Controller
- Views
- Passing Data to View
- Blade Templating Engine
- Display Multiple Data using Looping

TODAY's OUTLINE

- Migrations
- Eloquent ORM
- Models
- Accessor & Mutator

Database Preparation



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders 'storage', 'tests', and 'vendor', and files '.editorconfig', '.env', '.env.example', and '.gitattributes'. The '.env' file is selected and its contents are displayed in the code editor. The code editor has a dark theme and shows the following configuration for a MySQL database:

```
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=umn_genap2122_pemweb_w10_jetlive
15 DB_USERNAME=root
16 DB_PASSWORD=
```

Migrations: <https://laravel.com/docs/9.x/migrations>

Migrations are like version control for your database, allowing your team to define and share the application's database schema definition. If you have ever had to tell a teammate to manually add a column to their local database schema after pulling in your changes from source control, you've faced the problem that database migrations solve.

Generating Migrations:

```
php artisan make:migration create_flights_table
```

Migration Structure

A migration class contains two methods: **up** and **down**. The **up** method is used to add new tables, columns, or indexes to your database, while the **down** method should reverse the operations performed by the up method.

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('flights', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('airline');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::drop('flights');
    }
};
```

Migration Command

- Running Migration: `php artisan migrate`
- View Migration Status: `php artisan migrate:status`
- Rolling Back Migration: `php artisan migrate:rollback`
- Roll Back All Migrations: `php artisan migrate:reset`
- Roll Back & Migrate: `php artisan migrate:refresh`
- Drop All Tables & Migrate: `php artisan migrate:fresh`

Raw SQL Queries

```
DB::insert( query: "INSERT INTO posts(title, content) VALUES(?,?)",  
            ['PHP with Laravel', 'Laravel is the best thing that happen to PHP']);
```

```
$results = DB::select( query: "SELECT * FROM posts WHERE id = ?", [1]);
```

```
$updated = DB::update( query: "UPDATE posts SET title = ? WHERE id = ?", ["Updated title", 1]);
```

```
$deleted = DB::delete( query: "DELETE FROM posts WHERE id = ?", [1]);
```


Eloquent: <https://laravel.com/docs/9.x/eloquent>

Laravel includes Eloquent, an object-relational mapper (ORM) that makes it enjoyable to interact with your database. When using Eloquent, each database table has a corresponding "Model" that is used to interact with that table. In addition to retrieving records from the database table, Eloquent models allow you to insert, update, and delete records from the table as well.

Generating Model Classes

To get started, let's create an Eloquent model. Models typically live in the `app\Models` directory and extend the `Illuminate\Database\Eloquent\Model` class. You may use the `make:model` [Artisan command](#) to generate a new model:

```
php artisan make:model Flight
```

If you would like to generate a [database migration](#) when you generate the model, you may use the `--migration` or `-m` option:

```
php artisan make:model Flight --migration
```

Retrieving Models

```
use App\Models\Flight;

foreach (Flight::all() as $flight) {
    echo $flight->name;
}
```

Building Queries

The Eloquent `all` method will return all of the results in the model's table. However, since each Eloquent model serves as a query builder, you may add additional constraints to queries and then invoke the `get` method to retrieve the results:

```
$flights = Flight::where('active', 1)
    ->orderBy('name')
    ->take(10)
    ->get();
```

Inserting Models

Alternatively, you may use the `create` method to "save" a new model using a single PHP statement. The inserted model instance will be returned to you by the `create` method:

```
use App\Models\Flight;

$flight = Flight::create([
    'name' => 'London to Paris',
]);
```

However, before using the `create` method, you will need to specify either a `fillable` or `guarded` property on your model class. These properties are required because all Eloquent models are protected against mass assignment vulnerabilities by default. To learn more about mass assignment, please consult the [mass assignment documentation](#).

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Models\Flight;
use Illuminate\Http\Request;

class FlightController extends Controller
{
    /**
     * Store a new flight in the database.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
        // Validate the request...

        $flight = new Flight;

        $flight->name = $request->name;

        $flight->save();
    }
}
```

Mass Assignment

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The attributes that are mass assignable.
     *
     * @var array
     */
    protected $fillable = ['name'];
}
```

Once you have specified which attributes are mass assignable, you may use the `create` method to insert a new record in the database. The `create` method returns the newly created model instance:

```
$flight = Flight::create(['name' => 'London to Paris']);
```

Updating Models

```
use App\Models\Flight;

$flight = Flight::find(1);

$flight->name = 'Paris to London';

$flight->save();
```

Mass Updates

Updates can also be performed against models that match a given query. In this example, all flights that are **active** and have a **destination** of **San Diego** will be marked as delayed:

```
Flight::where('active', 1)
    ->where('destination', 'San Diego')
    ->update(['delayed' => 1]);
```

The **update** method expects an array of column and value pairs representing the columns that should be updated. The **update** method returns the number of affected rows.

Deleting Models

To delete a model, you may call the `delete` method on the model instance:

```
use App\Models\Flight;

$flight = Flight::find(1);

$flight->delete();
```

You may call the `truncate` method to delete all of the model's associated database records. The `truncate` operation will also reset any auto-incrementing IDs on the model's associated table:

```
Flight::truncate();
```

Deleting An Existing Model By Its Primary Key

In the example above, we are retrieving the model from the database before calling the `delete` method. However, if you know the primary key of the model, you may delete the model without explicitly retrieving it by calling the `destroy` method. In addition to accepting the single primary key, the `destroy` method will accept multiple primary keys, an array of primary keys, or a collection of primary keys:

```
Flight::destroy(1);

Flight::destroy(1, 2, 3);

Flight::destroy([1, 2, 3]);

Flight::destroy(collect([1, 2, 3]));
```

Soft Deleting

Soft Deleting

In addition to actually removing records from your database, Eloquent can also "soft delete" models. When models are soft deleted, they are not actually removed from your database. Instead, a `deleted_at` attribute is set on the model indicating the date and time at which the model was "deleted". To enable soft deletes for a model, add the `Illuminate\Database\Eloquent\SoftDeletes` trait to the model:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Flight extends Model
{
    use SoftDeletes;
}
```

Soft Delete: Database Table

You should also add the `deleted_at` column to your database table. The Laravel [schema builder](#) contains a helper method to create this column:

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::table('flights', function (Blueprint $table) {
    $table->softDeletes();
});

Schema::table('flights', function (Blueprint $table) {
    $table->dropSoftDeletes();
});
```

Now, when you call the `delete` method on the model, the `deleted_at` column will be set to the current date and time. However, the model's database record will be left in the table. When querying a model that uses soft deletes, the soft deleted models will automatically be excluded from all query results.

Accessor

An accessor transforms an Eloquent attribute value when it is accessed. To define an accessor, create a protected method on your model to represent the accessible attribute. This method name should correspond to the "camel case" representation of the true underlying model attribute / database column when applicable.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the user's first name.
     *
     * @return \Illuminate\Database\Eloquent\Casts\Attribute
     */
    protected function firstName(): Attribute
    {
        return Attribute::make(
            get: fn ($value) => ucfirst($value),
        );
    }
}
```

Mutator

A mutator transforms an Eloquent attribute value when it is set. To define a mutator, you may provide the set argument when defining your attribute.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Interact with the user's first name.
     *
     * @param string $value
     * @return \Illuminate\Database\Eloquent\Casts\Attribute
     */
    protected function firstName(): Attribute
    {
        return Attribute::make(
            get: fn ($value) => ucfirst($value),
            set: fn ($value) => strtolower($value),
        );
    }
}
```

SUMMARY

After finishing this module, you should be able:

- to understand how to implement Migrations
- to understand how to implement Eloquent ORM
- to understand how to implement Models
- to understand how to implement Accessor & Mutator

NEXT WEEK'S OUTLINE



- Eloquent Relationship
 - 1-to-1 Relationship
 - 1-to-many Relationship
 - Many-to-many Relationship

REFERENCES

<https://laravel.com/docs/9.x>

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.